

1.What is the output of the following, and why?

```
s = "Python"
Print(s[0], s[-1], s[6])
```

O/P: Error

Explanation:

*s[0]-> "P"

*s[-1]-> "n"

*s[6]-> invalid

*The string "Python" contains 6 characters and highest value of index is 5 so "s[6]" raise error "string index out of range".

2.What is the difference between positive and negative indexing? Give s="Hello", What does s[-2] return?

O/P: Positive indexing starts from left (0), while negative indexing starts from right (-1).

```
>s = "Hello"
```

```
>s[-2] → "l"
```

3. What happens when you access an index that is out of range in a string? How is this different from slicing out of range?

```
s = "Code"
print(s[10]) #vs
print(s[1:10])
```

O/P: *Accessing an index out of range gives index error.

*slicing out of range does not give error, it returns available characters only.

4. How do you find the index of the first and occurrence of a character in a string using indexing concepts?

```
s = "mississippi"
#Find index of first 's' and last 's'
```

O/P: Characters → m i s s i s s i p p i

Index value → 0 1 2 3 4 5 6 7 8 9

*Index of first 's'=2

*index of last 's'=6

5. What is the output of the following, and explain how negative indexing resolves?

```
s = "abcdef"
print(s[-1], s[-3], s[-6])
```

O/P: *Negative index count from right to left.

character → a b c d e f
index value → -6 -5 -4 -3 -2 -1

s[-1] = "f"
s[-3] = "d"
s[-6] = "a"

6. What is the syntax for string slicing in Python? Explain each part of s(start:stop:step).

O/P: Syntax → s[Start : Stop : Step]

Start = Starting index
Stop = Ending index(Not included)
Step = How many position to move

→ start value is smaller than stop value because slicing moves left → right

7. What if the output of the following slices, and why?

```
s="HelloWorld"
print(s[0:5]) → Hello
print(s[5:]) → World
print(s[:5]) → Hello
print(s[:]) → Helloworld
```

8. What happens when start is greater than stop in a slice with a positive step? What is the output?

```
s = "Python"
print(s[4:1])
```

O/P: If start is greater than stop and step is positive, output will be empty. Because Python cannot move forward from 4 to 1 with positive step.

9. How do you extract the last 3 characters of a string using slicing, without knowing its length?

O/P:
print(s[-3:])

Explanation:

→ -3 means starts from third character from the end.
→ : means take everything till the end of the string.

10. What is the difference between these two slices? What does each return?

```
s = "abcdefgh"
print(s[::1])
print(s[::2])
print(s[1::2])
```

O/P:

→ print(s[::1]) = abcdefgh (step = 1, takes every character)
→ print(s[::2]) = aceg (step = 2, index used = 0 2 4 6, takes every second character starting from index 0)
→ print(s[1::2]) = bdfh (starting index value = 1, step = 2, index used = 1 3 5 7)

11. How do you reverse a string using slicing? What is the step value that achieves this?

```
s = "Python"
# Reverse it using slicing
```

O/P: print(s[::-1])

→ step value -1 moves backward, so the string gets reversed.

12. What is the output of the following, and explain how the negative step works?

```
S = "abcdefgh"
Print(s[::-1]) → hgfedcba (step = -1, takes every character backwards)
Print(s[::2]) → hfdb (step = -2, moves backward by skipping one character each time)
Print(s[6:1:-1]) → gfedc (start = 6(g), stop = 1, move backward with step = -1)
```

13. How do you extract every 3rd character from a string starting from index 1?

```
S = "a1b2c3d4e5"
#expected output: "12345"
```

O/P: print(s[1::2])

→ starting index value = 1

→ step value = 2 (Takes every second character from index 1)

14. What is the output of the following negative step slices? Explain start and stop behavior when step is negative

```
s = "Python"
print(s[5:0:-1]) → nohty (start = 5, stop = 0, step = -1 moves backward)
print(s[5::-1]) → nohtyp (start = 5, goes till beginning)
print(s[0:-1]) → nohty (start from last index, stop before index 0, step = -1 move backward)
```

15. How do slices handle start and stop values that exceed string boundaries with a negative step?

```
S = "hello"  
print(s[100::-1])  
print(s[::-100])
```

Ans:

→ print(s[100::-1])

*start value 100 is outside the string boundary

*with negative step, Python automatically starts from the last character

O/P → olleh

→ print(s[::-100])

*step = 100, jumps 100 positions each time.

*since string length is small, only first character is taken.

O/P → h

16. How do you extract the middle portion of a string dynamically, regardless of its length?

```
s = "abcdefghij" #10 chars
```

```
#Extract characters from index 2 to second last(exclusive)
```

Ans:

```
print(s[2:-1]) → O/P: cdefghi
```

* Start from index 2

* -1 means last character

* Stop is exclusive, so last character is not included

17. How do you use slicing to check if a string is a palindrome in a single line?

```
s = "racecar"
```

```
# Check palindrome using slicing
```

Ans: print(s == s[::-1])

* s[::-1] reverses the string

* If original string and reversed string are same, it is a palindrome

18. What is the output of the following, and why does python not raise an error?

```
s = "hello"
```

```
print(s[1:100])
```

```
print(s[-100:3])
```

```
print(s[100:200])
```

Ans: 1. `s[1:100]` → O/P: ello (start = 1, stop = 100)

2. `s[-100:3]` → O/P: hel (-100 is smaller than valid range, Python starts from beginning. Takes characters before index 3)

3. `Print(s[100:200])` → O/P: empty

→ Python slicing does not raise an error when indexes are out the string range. It automatically adjusts the boundaries.

19. How do you replace a portion of a string using slicing (since strings are immutable)?

```
S = "Hello World"
```

```
#Replace "world" with "Python" using slicing and concatenation
```

Ans:

```
s = "Hello World"
```

```
new_s = s[:6] + "Python"
```

```
print(new_s)
```

→ O/P: Hello Python

Explanation:

*`s[:6]` → "Hello"

*Concatenate with "Python"

*Strings are immutable, so a new string is created instead of modifying the original string.

20. How does Python's `slice()` object work, and how can you use it as a reusable slicer?

```
s = "abcdefghij"
```

```
My_slice = slice(1,8,2)
```

```
print(s[My_slice])
```

```
#what is the output, and how can this be reused on other strings?
```

Ans:

→ O/P : bdfh (`slice(1, 8, 2)` means start = 1, stop = 8, step = 2)

The same slice object can be reused on other strings.

ex:

```
my_slice= slice(1, 8, 2)
```

```
print("123456789"[my_slice]) → O/P: 2468
```

→ `slice()` stores slicing rules and can be reused multiple times.